University of Asia Pacific
Department of Computer Science & Engineering

Page Replacement

FIFO vs MRU — page faults and hits compared on the same reference string

FIFO · 8 FAULTS, 1 HIT



MRU · 6 FAULTS, 3 HITS



EACH BLOCK = ONE PAGE REFERENCE · AMBER = PAGE FAULT · TEAL = HIT · 3 FRAMES

9	3	MRU	2
REFERENCES	FRAMES	FEWER FAULTS	FAULTS SAVED

COURSE	CSE 402 — Operating System Lab
EXPERIMENT	Page Replacement — FIFO & MRU
REPOSITORY	github.com/ashadsetu/CSE-402

SUBMITTED BY

MD Asadullah
Reg. No 23101126 · Section C1
4th Year, 1st Semester

SUBMITTED TO

Atia Rahman Orthi
Lecturer, Department of CSE
University of Asia Pacific

PROBLEM

Run a page reference string through a fixed set of frames with two replacement policies and compare how many page faults and hits each produces. **FIFO** evicts the page that has been in memory longest; **MRU** evicts the page used most recently.

INPUT	VALUE
Reference string	1, 2, 3, 4, 5, 1, 2, 3, 1
Frames	3

CODE

```
# FIFO
pages =[1,2,3,4,5,1,2,3,1]
capacity = 3

frames =[]
faults = 0
hits = 0

for page in pages:
    if page not in frames:
        faults += 1
        if len(frames) < capacity:
            frames.append(page)
        else:
            frames.pop(0)
            frames.append(page)
    else:
        hits += 1
    print("Page:", page, "Frames:", frames)

total = len(pages)
fault_ratio = faults/ total
hit_ratio = hits/ total
fault_hit_ratio = hits/ faults


print("Total Faults:", faults)
print("Total Hits:", len(pages) - faults)
print("Page fault ratio:", fault_ratio)
print("Page hit ratio", hit_ratio)
print("Fault to hit ratio", fault_hit_ratio)

# MRU
pages =[1,2,3,4,5,1,2,3,1]
capacity = 3

frames =[]
faults = 0
hits = 0

for page in pages:
    if page not in frames:
        faults += 1
        if len(frames) < capacity:
            frames.append(page)
        else:
            frames.pop(-1)
            frames.append(page)
    else:
        hits += 1
        frames. remove(page)
        frames.append(page)
    print("Page:", page, "Frames:", frames)

total = len(pages)
fault_ratio = faults/ total
hit_ratio = hits/ total
fault_hit_ratio = hits/ faults

print("Total Faults:", faults)
print("Total Hits:", hits)
print("Page fault ratio:", fault_ratio)
print("Page hit ratio:", hit_ratio)
print("Fault to hit ratio", fault_hit_ratio)
```

RESULTS

RESULT — FIFO

STEP	PAGE	FRAMES	RESULT
1	1	[1]	Fault
2	2	[1, 2]	Fault
3	3	[1, 2, 3]	Fault
4	4	[2, 3, 4]	Fault
5	5	[3, 4, 5]	Fault
6	1	[4, 5, 1]	Fault
7	2	[5, 1, 2]	Fault
8	3	[1, 2, 3]	Fault
9	1	[1, 2, 3]	Hit

```
Total Faults: 8
Total Hits: 1
Page fault ratio: 0.8888888888888888
Page hit ratio 0.1111111111111111
Fault to hit ratio 0.125
```

RESULT — MRU

STEP	PAGE	FRAMES	RESULT
1	1	[1]	Fault
2	2	[1, 2]	Fault
3	3	[1, 2, 3]	Fault
4	4	[1, 2, 4]	Fault
5	5	[1, 2, 5]	Fault
6	1	[2, 5, 1]	Hit
7	2	[5, 1, 2]	Hit
8	3	[5, 1, 3]	Fault
9	1	[5, 3, 1]	Hit

```
Total Faults: 6
Total Hits: 3
Page fault ratio: 0.6666666666666666
Page hit ratio: 0.3333333333333333
Fault to hit ratio 0.5
```

VERDICT

METRIC	FIFO	MRU
Page faults	8	6
Page hits	1	3
Fault ratio	0.889	0.667
Hit ratio	0.111	0.333

MRU
6 faults · 3 hits

FIFO
8 faults · 1 hit

MRU has two fewer faults and three times the hits on this reference string.

DISCUSSION

- **FIFO evicts exactly the pages it needs next.** The string loops back to 1, 2, 3 after 4 and 5, but FIFO has already thrown out the oldest pages to make room, so 1, 2 and 3 all fault again. Its only hit is the last reference to 1.
- **MRU protects the old pages.** When 4 and 5 arrive it evicts the newest page in the frames (3, then 4), so 1 and 2 stay resident and hit on their return. It still faults on the second 3, which it had evicted earlier.
- **This is a best case for MRU, not a general rule.** It works here because the string is a loop larger than memory, where the most recent page is the one least likely to come back. With ordinary locality, recently used pages are reused soon, and MRU would evict the wrong ones; that is why LRU is the usual choice. FIFO, in contrast, needs no usage tracking at all.
- **One label in the scripts doesn't match its formula.** "Fault to hit ratio" is computed as `hits / faults`, so it is really a hit-to-fault ratio (0.125 for FIFO, 0.5 for MRU). The totals above aren't affected.

CONCLUSION

- On the same reference string and three frames, **MRU gave 6 faults against 8** for FIFO, lifting the hit ratio from 0.111 to 0.333.
- The gap comes from the loop pattern: FIFO keeps evicting pages that are about to be reused, while MRU keeps them in memory.
- MRU's advantage is specific to looping access; for typical workloads, a policy based on least recent use is the safer choice, and FIFO remains the simplest to implement.

`MRU wins here — 6 faults vs 8 — because the reference string loops past the frame count.`